

Bases de Datos Columnares

NSM vs. DSM, compresión

IIC2413 · Semana 16

La pregunta de la semana pasada

Al final de la Semana 15 vimos que incluso con el star schema, las consultas analíticas sobre millones de filas son lentas.

Pregunta que quedó abierta:

¿Por qué una consulta que solo necesita 2 columnas de una tabla de 50 columnas tiene que leer páginas enteras?

La respuesta está en cómo los datos están organizados en disco.

La idea de fondo: aplanar una tabla

Una tabla es **2D** (filas × columnas). El disco es **1D**: una secuencia de bytes. Para guardarla hay que **elegir un orden**.

Tabla:	id	nombre	ciudad
	1	Ana	Stgo
	2	Luis	Valp

Por filas (NSM): 1,Ana,Stgo | 2,Luis,Valp | ...
Por columnas (DSM): 1,2,... | Ana,Luis,... | Stgo,Valp,...

- **Por filas:** todo lo de una fila, junto → bueno para "dame esta fila".
- **Por columnas:** todo lo de una columna, junto → bueno para "dame esta columna de todas las filas".

Misma tabla, dos formas de aplanarla. Toda esta semana es **esta única decisión** y sus consecuencias.

NSM: N-ary Storage Model (row store)

En NSM, cada página contiene filas completas:

Página k (8 KB):

id	nombre	email	ciudad	42 cols	...	fecha_nac
id	nombre	email	ciudad	42 cols	...	fecha_nac
id	nombre	email	ciudad	42 cols	...	fecha_nac

Una tupla completa entra en la página, aunque la consulta solo pida 1 columna.

Consecuencia:

```
SELECT nombre FROM dim_cliente;  -- necesita 1 de 50 columnas
```

→ El DBMS carga **todas** las páginas del heap, incluyendo las 49 columnas que no necesita.

DSM: Decomposition Storage Model (column store)

En DSM, cada columna ocupa sus propias páginas:

```
Páginas de col_id:      [1][2][3][4][5][6] ... [1.000.000]
Páginas de col_nombre:  ['Ana']['Luis']['María'] ... ['Pedro']
Páginas de col_email:   ['ana@...']['luis@...'] ... ['pedro@...']
Páginas de col_ciudad:  ['Stgo']['Valp'] ... ['Conce']
... (46 columnas más)
```

Ahora:

```
SELECT nombre FROM dim_cliente;
```

→ El DBMS carga **solo** las páginas de `col_nombre`.

Las 49 columnas restantes **no se leen del disco en absoluto**.

El cálculo: la intuición

`SELECT a, b FROM dim_cliente` necesita **2 columnas de 50**.

- **NSM:** esas 2 columnas viven dentro de filas completas → para leerlas hay que traer **toda la tabla** del disco.
- **DSM:** esas 2 columnas son 2 archivos → traés **solo esos 2**, los otros 48 ni se tocan.

Pagás I/O por lo que **tocás**, no por lo que **existe**. Todo depende de qué fracción de las columnas necesita tu consulta.

El cálculo: los números

Tabla `dim_cliente`: **1M filas, 50 columnas** de 8 bytes. Páginas de 8 KB.

NSM — todo junto:

- $1.000.000 \times 50 \times 8 = 400 \text{ MB} = \mathbf{50.000 \text{ páginas}}$
- `SELECT a, b` → lees las **50.000** (no hay forma de saltar las otras 48 columnas)

DSM — una columna por archivo:

- cada columna = $1.000.000 \times 8 = 8 \text{ MB} = \mathbf{1.000 \text{ páginas}}$
- `SELECT a, b` → lees **2.000 páginas**

Modelo	Páginas leídas	
NSM	50.000	—
DSM	2.000	25x menos I/O

NSM vs. DSM: trade-off

Todo se reduce a **una** pregunta: ¿optimizo *"dame esta fila"* o *"dame esta columna de todas las filas"*?

	NSM (row store)	DSM (column store)
Layout	filas juntas	columnas separadas
Leer pocas filas / por PK	✓ una página	× N páginas
Escanear pocas columnas	× lee filas enteras	✓ solo esas columnas
Insertar / actualizar una fila	✓ 1 página	× N páginas (una por columna)
Compresión	regular (heterogéneo)	excelente (homogéneo)
Uso ideal	OLTP	OLAP

Por el costo de insert, columnar se usa principalmente para analítica.

¿Por qué la compresión impacta más en DSM?

En NSM, una página tiene valores de **distintos tipos**:

```
[INT] [VARCHAR] [FLOAT] [DATE] [BOOLEAN] [INT] [...]
```

Muy difícil comprimir: los valores son heterogéneos.

En DSM, una página tiene valores del **mismo atributo**:

```
[Norte] [Norte] [Norte] [Sur] [Sur] [Sur] [Sur] [Centro] [Centro]
```

Los valores son del mismo dominio → **mucha repetición** → compresión muy efectiva.

¿Por qué la compresión impacta más en DSM?

En OLAP con DSM, la compresión reduce:

1. El espacio en disco (→ datos caben en la nube más barato)
2. El I/O de lectura (→ se leen menos bytes físicos)
3. El uso de memoria (→ más datos caben en buffer pool)

Cuatro trucos para comprimir.

Los valores de una columna **se parecen entre sí** (mismo tipo, mismo dominio, a veces ordenados). Cada técnica explota una forma distinta de ese parecido:

Truco	Explota...	Ideal para
RLE	valores repetidos seguidos	columnas ordenadas
Dictionary	pocos valores distintos	strings que se repiten
Bitmap	baja cardinalidad	categorías, filtros
Delta	valores cercanos entre sí	fechas, ids, contadores

El motor elige lo que más comprime.

Compresión 1: Run-Length Encoding (RLE)

Idea: representar secuencias repetidas como `(valor, cantidad)`.

Columna region (sin comprimir):

Norte | Norte | Norte | Norte | Sur | Sur | Centro | Centro | Centro

Columna region (RLE):

(Norte, 4) | (Sur, 2) | (Centro, 3)

Ahorro: $9 \text{ valores} \times 4 \text{ bytes} = 36 \text{ bytes} \rightarrow 3 \text{ entradas} \times 8 \text{ bytes} = 24 \text{ bytes}$.

Compresión 1: Run-Length Encoding (RLE)

Columna region (sin comprimir):

Norte | Norte | Norte | Norte | Sur | Sur | Centro | Centro | Centro

Columna region (RLE):

(Norte, 4) | (Sur, 2) | (Centro, 3)

Potencia real en OLAP: si la columna está ordenada por `region` (cosa común en un DW), las secuencias son largas:

(Norte, 400.000) | (Sur, 350.000) | (Centro, 250.000)

Y la consulta `WHERE region = 'Norte'` → **ni siquiera escanea la columna:** es un rango `[0, 400.000)` en el RLE.

RLE sobre columnas ordenadas es el ejemplo más poderoso de "operar sobre datos comprimidos sin descomprimir".

Compresión 2: Dictionary Encoding

Idea: reemplazar strings repetidos por enteros pequeños. El diccionario se guarda una sola vez.

```
Columna categoria (sin comprimir): 11 bytes × N filas  
'Electrónica' | 'Ropa' | 'Electrónica' | 'Alimentos' | 'Ropa'
```

```
Diccionario: 0 → 'Electrónica'  
             1 → 'Ropa'  
             2 → 'Alimentos'
```

```
Columna (comprimida): 2 bits × N filas  
0 | 1 | 0 | 2 | 1
```

Con 3 categorías distintas → 2 bits por valor (en vez de 11 bytes).

Ahorro: ~44x.

Compresión 3: Bitmap Encoding

Idea: para columnas de **baja cardinalidad** (pocas categorías distintas), crear un bit-vector por cada valor posible.

Columna region:	Norte	Sur	Norte	Centro	Norte	Sur	
Bitmap Norte:	1	0	1	0	1	0	(6 bits)
Bitmap Sur:	0	1	0	0	0	1	
Bitmap Centro:	0	0	0	1	0	0	

Compresión 3: Bitmap Encoding

Columna region: Norte | Sur | Norte | Centro | Norte | Sur

Bitmap Norte: 1 | 0 | 1 | 0 | 1 | 0 (6 bits)

Bitmap Sur: 0 | 1 | 0 | 0 | 0 | 1

Bitmap Centro: 0 | 0 | 0 | 1 | 0 | 0

Operaciones booleanas directas:

WHERE region = 'Norte' → usar Bitmap Norte directamente

WHERE region IN ('Norte', 'Sur') → OR bit a bit de los dos bitmaps

WHERE region = 'Norte' AND año = 2025 → AND de dos bitmaps

Intersección de dos predicados sobre 1 millón de filas → **una sola pasada AND** sobre dos arrays de bits de 128 KB.

Compresión 4: Delta Encoding

Idea: en columnas ordenadas con valores cercanos, almacenar las **diferencias** en lugar de los valores absolutos.

```
Columna fecha (timestamp Unix):  
1704067200 | 1704067260 | 1704067320 | 1704067380  
  
Delta:  
1704067200 | +60 | +60 | +60
```

El valor inicial ocupa 4 bytes; los deltas ocupan 1 byte. Ahorro: ~4x.

También sirve para `id` secuenciales o precios con variación pequeña.

Operar sobre datos comprimidos

La innovación clave de los sistemas columnar no es solo comprimir — es **no descomprimir para responder consultas**.

Operación	Sin compresión	Con compresión
WHERE region = 'Norte'	Scan de strings	Lookup en diccionario, scan de códigos enteros
COUNT(*) WHERE region = 'Norte'	Scan completo	Leer RLE: (Norte, 400.000) → resultado inmediato
SUM(monto) WHERE region IN (...)	Scan + filtro	Bitmap OR, luego SUM solo sobre las posiciones marcadas

Regla de oro: en un sistema columnar bien implementado, la mayoría de las consultas OLAP *ni siquiera miran los datos originales* — operan directamente sobre las representaciones comprimidas.

El límite del columnar "puro"

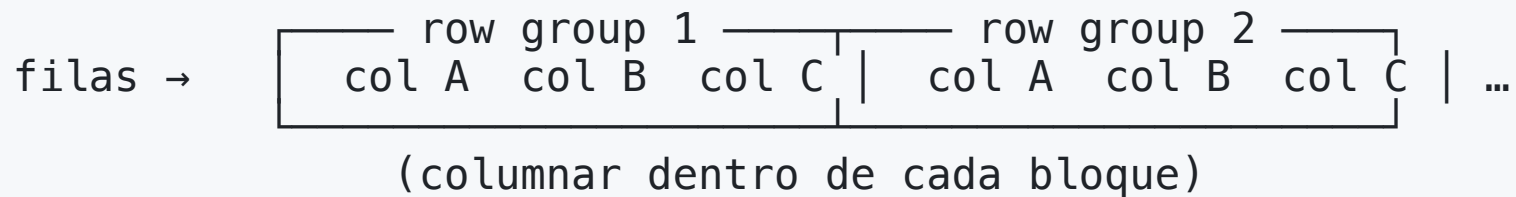
DSM puro —cada columna en su propio archivo gigante— tiene costos prácticos:

- **Reconstruir una fila** exige saltar al índice i en N archivos distintos.
- **No hay dónde "saltar"**: un archivo-columna no tiene estructura interna; para coordinar valores de dos atributos distintos hay que escanear casi todo.
- **Append**: agregar filas toca *todos* los archivos de columna.

Queremos las ventajas del columnar (leer pocas columnas, comprimir bien) **sin** estos costos. La propuesta es un layout **híbrido**.

La idea de Parquet: híbrido por row groups

Parquet parte la tabla en **row groups** (bloques de filas) y, *dentro* de cada uno, guarda los datos **por columna**.



- Cada row group es **autocontenido**: se lee, comprime, salta y procesa de forma **independiente**.
- Lo mejor de dos mundos: *dentro*, columnas homogéneas (proyección + compresión); *entre* bloques, unidades que un motor puede **repartir** entre workers o **descartar** con metadata.
- Un **footer** al final describe el schema y trae estadísticas por columna por row group.

Parquet: el formato columnar estándar

Parquet es un formato de archivo columnar abierto, el estándar de facto del ecosistema de datos.

```
archivo.parquet
├── File Metadata (footer)           ← schema, estadísticas, offsets
│   ├── schema: id INT, nombre STRING, ciudad STRING, monto FLOAT
│   ├── num_rows: 1.000.000
│   └── ...
├── Row Group 1 (~128 MB)
│   ├── col_id: [valores comprimidos, RLE/Delta]
│   ├── col_nombre: [valores comprimidos, Dictionary]
│   ├── col_ciudad: [valores comprimidos, Dictionary + RLE]
│   └── col_monto: [valores comprimidos, Delta]
├── Row Group 2 (~128 MB)
├── ...
└── Row Group N
```

Row group: las filas se dividen en grupos de ~128 MB. Dentro de cada grupo, el almacenamiento es columnar.

Parquet: por qué el footer importa tanto

El footer de Parquet contiene **estadísticas por columna por row group**:

- `min` / `max` de cada columna
- `null_count`
- Diccionarios de valores frecuentes

Esto permite **predicate pushdown** sin leer los datos:

```
# Consulta: SELECT * FROM 'ventas.parquet' WHERE año = 2023
```

```
# El motor lee solo el footer:
```

```
# Row Group 1: año min=2020, max=2022 → skip (no puede contener 2023)
```

```
# Row Group 2: año min=2023, max=2023 → leer
```

```
# Row Group 3: año min=2023, max=2024 → leer
```

```
# Row Group 4: año min=2025, max=2025 → skip
```

Parquet es inmutable: ¿y los cambios?

Un archivo Parquet **no se edita**: no hay `UPDATE` ni `DELETE` fila a fila. Pero las tablas reales cambian. ¿Cómo?

Idea clave: no tocar los archivos viejos. Guardar los cambios como **archivos nuevos** — *deltas*.

- **Insertar** filas → escribir un **nuevo** Parquet.
- **Borrar / actualizar** → un pequeño **delete file** que marca "estas filas ya no cuentan", sin reescribir el archivo original.

Una tabla deja de ser *un archivo* y pasa a ser *un conjunto de archivos que va creciendo*.

Table formats: Parquet + un log de cambios

Agregamos los cambios a parquet, con la misma estructura (incluidos footer). Eso, sobre object storage barato, da:

- **ACID** y *snapshot isolation*: lecturas consistentes aunque alguien esté escribiendo.
- **Time travel**: consultar la tabla "como era" en una versión anterior — el log la reconstruye.

Delta Lake (Databricks) y **Apache Iceberg** (Netflix) son los dos estándares: *Parquet + una capa de metadatos que registra los cambios*.

Resumen

- **NSM (row store)**: filas juntas → óptimo para OLTP.
- **DSM (column store)**: una columna por archivo → lees solo lo que tocás ($\approx 25\times$ menos I/O en OLAP).
- **Compresión** (columnas homogéneas): RLE, Dictionary, Bitmap, Delta — y se **opera sobre los datos comprimidos**, sin descomprimir.
- **Parquet**: layout híbrido (row groups + columnar dentro) + footer de estadísticas → proyección, compresión y *predicate pushdown* en un archivo portable.
- **Table formats** (Delta Lake / Iceberg): Parquet + log de cambios → `UPDATE` / `DELETE`, ACID y *time travel*.

Bases de Datos Columnares

Late Materialization, Vectorización y Sistemas

El problema: volver a armar las filas

DSM guarda las columnas por separado, pero una consulta pide **filas**:

```
SELECT nombre, email
FROM dim_cliente
WHERE ciudad = 'Santiago';
```

Esto podemos hacerlo: las columnas están **alineadas por posición**. La fila i es el i -ésimo valor de `col_ciudad`, de `col_nombre` y de `col_email`.

La pregunta: ¿en qué momento volvemos a juntar las columnas en filas?

- **Early materialization:** lo antes posible (apenas empieza el plan).
- **Late materialization:** lo más tarde posible (después de filtrar).

Early Materialization: enfoque ingenuo

```
Para cada fila i:  
  1. Lee col_ciudad[i]  
  2. Si ciudad = 'Santiago':  
    a. Lee col_nombre[i]      ← acceso a otra columna  
    b. Lee col_email[i]      ← acceso a otra columna  
    c. Emite la tupla (nombre, email)
```

Problema: para cada fila que **pasa el filtro**, debemos acceder a las otras columnas.

En acceso aleatorio, cada `col_nombre[i]` y `col_email[i]` puede caer en una página diferente.

Early materialization **elimina la ventaja de DSM** al obligar accesos aleatorios en otras columnas.

Late materialization (1): trabajar con posiciones

La idea: no muevas valores, mueve posiciones.

Fase 1 — filtrar `ciudad`, producir una *position list*:

```
col_ciudad:  i=0 'Santiago' ✓   i=1 'Valparaíso' x   i=2 'Santiago' ✓ ...  
position list → [0, 2, 5, 8, 11, ...]      (solo los índices que pasan)
```

Fase 2 — refinar con los otros filtros, sobre esas posiciones:

```
+ WHERE año = 2025 → mirar col_año solo en [0, 2, 5, 8, 11, ...]  
position list refinada → [0, 5, 11, ...]
```

Hasta acá movimos solo **índices**. No tocamos `nombre` ni `email`.

Late materialization (2): materializar al final

Fase 3 — recién ahora traer las columnas de proyección, solo en las posiciones que sobrevivieron:

```
fetch col_nombre en [0, 5, 11, ...] → ['Ana', 'Luis', 'María', ...]  
fetch col_email  en [0, 5, 11, ...] → ['ana@...', 'luis@...', ...]
```

La fila se reconstruye **lo más tarde posible**, y solo para las filas que de verdad importan.

Si 80.000 de 1.000.000 de filas pasan el filtro, traemos `nombre / email` **80.000** veces, no un millón.
Cuanto más selectivo el filtro y más columnas la tabla, más gana.

Por qué late materialization gana en I/O

```
SELECT nombre, email
FROM dim_cliente
WHERE ciudad = 'Santiago' AND año_alta >= 2020;
```

Tabla: **1 millón de filas**, 50 columnas.
Filas que pasan ambos filtros: **80.000** (8%).

Estrategia	Columnas leídas	Filas accedidas
Early	col_ciudad + col_año + col_nombre + col_email	1.000.000 × 4 cols
Late	col_ciudad + col_año (scan completo)	1.000.000 × 2 cols
	col_nombre + col_email (solo matches)	80.000 × 2 cols

Late materialization: ~2.160.000 accesos de valor
Early materialization: ~4.000.000 accesos de valor

Ejecución vectorizada: una tupla vs. un vector a la vez

Recordar: **pipeline de operadores** que piden datos con un iterador tipo `next()`. Pero **cuántos datos** entrega cada `next()` ?.

Iterador (Volcano) — una tupla a la vez:

```
for i in range(N):           # N millones de veces
    fila = scan.next()       # sube 1 tupla por todo el pipeline
    if filter(fila):         # 1 comparación de trabajo útil
        aggregate.add(fila)
```


Ejecución vectorizada: una tupla vs. un vector a la vez

Vectorizado — un vector (~2048 valores) a la vez:

```
for chunk in scan.next_vector():      # N/2048 veces
    mask = chunk.edad > 30             # 2048 comparaciones en un loop apretado
    aggregate.add_vector(chunk[mask])
```

El overhead se amortiza sobre ~2048 valores → loop contiguo y **SIMD-izable** (próxima slide).

Por qué es bueno CPUs + vectores (1): SIMD

SIMD = Single Instruction, Multiple Data: una instrucción que opera sobre **varios valores a la vez**.

Un registro moderno de 512 bits = 8 enteros de 64 bits:

```
Comparar  [1, 3, 2, 1, 3, 2, 1, 3] == 2  
Resultado [0, 0, 1, 0, 0, 1, 0, 0]  
          → 8 comparaciones en 1 sola instrucción
```

Por qué es bueno CPUs + vectores (2): cache y branches

Cache locality: un chunk de 1024 enteros de 8 bytes = 8 KB → cabe en la **L1 cache**. Se procesa entero antes de pedir el siguiente → acceso secuencial, sin esperar a memoria.

Branch prediction: sin una llamada de función por tupla, el loop interno es predecible → el CPU optimiza en vez de ir adivinando saltos.

Procesar una consulta columnar (1): agregación

```
SELECT region, AVG(edad)
FROM clientes
GROUP BY region;           -- archivo ordenado por region (común en un DW)
```

1. **Proyección:** leer solo `region` y `edad`; las otras 48 columnas no se tocan.
2. `region` **está en RLE** (ordenada): `(Norte, 400k) (Sur, 350k) (Centro, 250k)`. Cada run **es** un grupo y marca un rango de filas exacto → el `GROUP BY` es *slicing*, no hash.

Procesar una consulta columnar (1): agregación

```
SELECT region, AVG(edad)
FROM clientes
GROUP BY region;           -- archivo ordenado por region (común en un DW)
```

3. **AVG sobre datos comprimidos:** si `edad` viene en RLE/diccionario, se suma sobre las *runs*, sin expandir las filas. Para Norte:

```
edad (RLE):  (34, 100) (35, 150) (36, 50)
SUM          = 34·100 + 35·150 + 36·50 = 10.450
COUNT      = 100 + 150 + 50          = 300
AVG          = 10.450 / 300            ≈ 34,8    ← 3 runs, no 300 filas
```

Procesar una consulta columnar (1): agregación

```
SELECT region, AVG(edad)
FROM clientes
GROUP BY region;           -- archivo ordenado por region (común en un DW)
```

El `GROUP BY` y el `AVG` se calculan casi **sobre los metadatos de compresión**: tocan una fracción de los datos reales, y escalan igual con millones de filas y las mismas pocas runs.

Procesar una consulta columnar (2): un join

Los joins necesitan **valores reales** para emparejar llaves → no todo se queda comprimido.

```
SELECT c.region, SUM(h.monto)
FROM hechos h JOIN dim_cliente c ON h.id_cliente = c.id_cliente
WHERE c.region = 'Norte'
GROUP BY c.region;
```

1. **Proyección:** de `hechos`, solo `id_cliente` y `monto`; de `dim_cliente`, solo `id_cliente` y `region`.
2. **Pruning con metadata:** con el `min / max` de `id_cliente` por row group, **saltar** row groups de `hechos` sin posible match.
3. **Descomprimir las llaves:** para hashear hacen falta los `id_cliente` **reales** (decodificar el diccionario). Las llaves del join no se quedan comprimidas.

Procesar una consulta columnar (2): un join

Ya tenemos los row group y las llaves.

```
SELECT c.region, SUM(h.monto)
FROM hechos h JOIN dim_cliente c ON h.id_cliente = c.id_cliente
WHERE c.region = 'Norte'
GROUP BY c.region;
```

4. **Build / probe:** hash table con los `id_cliente` de `dim` que cumplen `region = 'Norte'`; *probe* con los de `hechos`
→ **pares de posiciones** que matchean.
5. **Late materialization:** recién ahora se hace *fetch* de `monto` en las posiciones que matchearon → `SUM`.

Un join sin hash: aprovechar el orden

El join anterior construyó una **hash table**. Pero recordemos un detalle del data warehouse: las tablas suelen estar **ordenadas por su llave** (la dimensión por su PK; los hechos por `id_fecha` o `id_cliente`).

Si **ambos** lados ya vienen ordenados por la llave del join, no hace falta hash: podemos hacer Merge join. .

```
hechos      (ord. por id_cliente):  1 1 1 2 3 3 ...
dim_cliente (ord. por id_cliente):  1 2 3 4 ...
```

↑ ↑

dos punteros que avanzan en paralelo, sin retroceder

Sort-Merge Join sobre columnas RLE

Y una columna ordenada se guarda **RLE-comprimida** (es lo que RLE comprime mejor).

```
col_key_R (RLE):  (A, 3) | (B, 2) | (C, 4)    ← A aparece 3 veces, ...  
col_key_S (RLE):  (A, 5) | (B, 1) | (D, 2)
```

Sort-Merge Join sobre columnas RLE

Avanzamos run contra run, igual que el merge clásico avanzaba fila contra fila:

Run de R	Run de S	¿match?	Resultado
(A, 3)	(A, 5)	✓	(A, $3 \times 5 = 15$)
(B, 2)	(B, 1)	✓	(B, $2 \times 1 = 2$)
(C, 4)	(D, 2)	×	—

Un run de 3 A contra un run de 5 A matchea: el resultado es el **producto cartesiano** de sus rangos de posiciones, $[\text{posR}..\text{posR}+3) \times [\text{posS}..\text{posS}+5)$. No se enumera: queda **factorizado** como ese par de rangos.

Encontrar **qué** runs matchean cuesta $O(\#runs)$: cardinalidad 3 sobre 1M filas son 3 runs → **6 comparaciones**, no 2 millones. El resultado sigue factorizado como rangos $\times$ rangos.

Sort-Merge Join sobre columnas RLE

Run de R	Run de S	¿match?	Resultado
(A, 3)	(A, 5)	✓	(A, $3 \times 5 = 15$)
(B, 2)	(B, 1)	✓	(B, $2 \times 1 = 2$)
(C, 4)	(D, 2)	×	—

- Su **cardinalidad** ($3 \times 5 = 15$) es una multiplicación, disponible al instante.
- Si arriba hay un `COUNT` / `SUM`, se calcula desde los largos de los runs — las 15 filas **nunca se expanden**.
- Solo si la consulta debe emitir filas una por una se expande el producto, y lo más tarde posible.

El problema del orden: ¿ordenar por cuál?

El SMJ es óptimo si la tabla de hechos está ordenada por la llave del join. Pero un star schema hace **varios joins a la vez**:

```
... WHERE dim_fecha.año      = 2025  
        AND dim_cliente.region = 'Norte'  
        AND dim_producto.familia = 'Electrónica'
```

Si ordenamos los hechos por `id_fecha`, ayudamos al join con `dim_fecha` — pero `id_cliente` e `id_producto` quedan desordenados.

Un orden lineal solo puede favorecer a una dimensión. Las otras vuelven a necesitar hash o random I/O.

¿Existe un orden que deje "cercanas en varias dimensiones a la vez" a las filas físicamente juntas?

Z-ordering: una curva que entrelaza dimensiones

Z-ordering (curva de Morton): intercalar los bits de varias llaves en **un solo número**, y ordenar la tabla por ese número.

```
id_fecha    = 0 1 1
id_cliente  = 1 0 0
              ↓ intercalar bit a bit
z           = 01 10 10 → un único valor de orden
```

Ordenar por **z** deja juntas las filas cercanas en **todas** las dimensiones: no es perfecto para ninguna, pero es bueno para todas a la vez.

Z-value: definición formal

Sea una fila con d atributos $(a_1, \dots, a_d)$, cada uno un entero de b bits. Su **z-value** intercala los bits: el bit j del atributo k va a la posición $j \cdot d + k$.

```
d = 2,  atributos x, y de 2 bits:  
  
x = x1 x0  
y = y1 y0  
z = y1 x1 y0 x0      ← bits intercalados (4 bits)
```

Z-value: definición formal

El **z-ordering** ordena las filas por su z-value. Esto recorre la grilla d -dimensional con una **curva de relleno (space-filling) en forma de "Z"** que preserva la vecindad: puntos cercanos en el espacio quedan (casi siempre) cercanos en **z**.

orden de visita de la grilla 4×4:

y=3	10	11	14	15
y=2	8	9	12	13
y=1	2	3	6	7
y=0	0	1	4	5
	x=0	1	2	3

cada bloque 2×2 es una "Z":

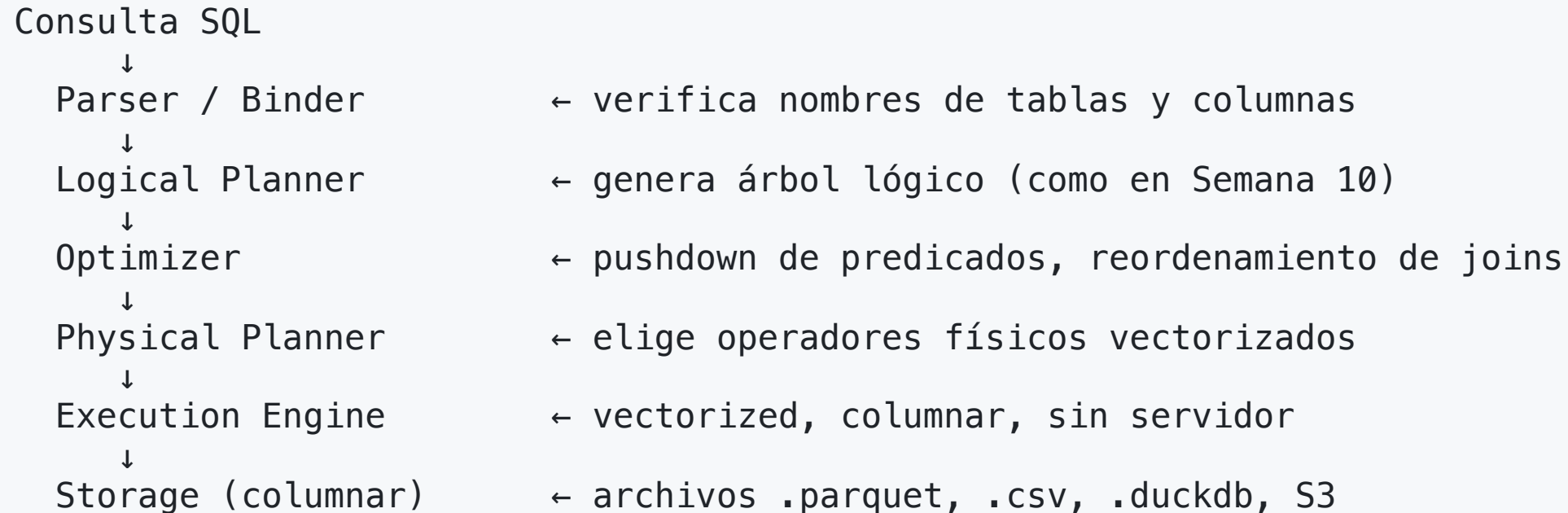
$(0,0) \rightarrow (1,0) \rightarrow (0,1) \rightarrow (1,1) = 0,1,2,3$

vecinos en 2D

→ z-values cercanos

→ mismas páginas / row groups

DuckDB: arquitectura de un sistema columnar embebido

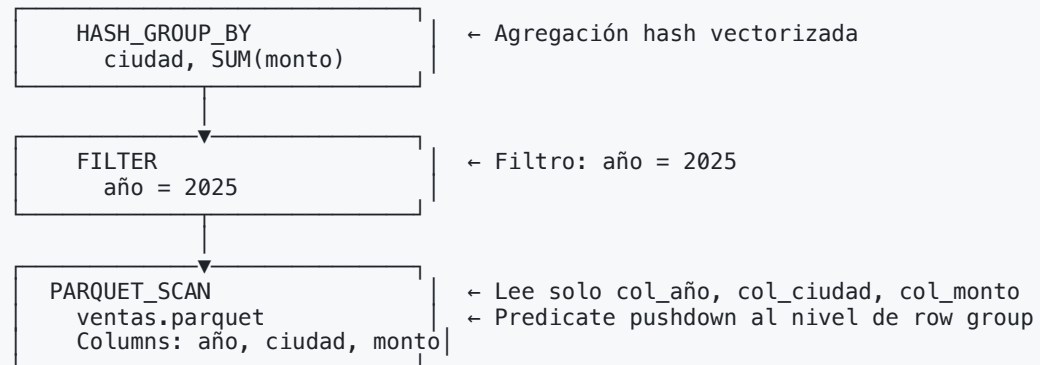


Sin servidor: no hay proceso separado. DuckDB corre en el mismo proceso Python.

Sin setup: solo `pip install duckdb`.

DuckDB: EXPLAIN y planes de ejecución

```
EXPLAIN SELECT ciudad, SUM(monto) FROM 'ventas.parquet'  
WHERE año = 2025 GROUP BY ciudad;
```



Comparación: EXPLAIN en PostgreSQL vs. DuckDB

PostgreSQL (row store, OLTP):

```
Seq Scan on ventas (cost=0.00..120000.00 rows=50000)
  Filter: (año = 2025)
  → Lee TODAS las páginas del heap aunque sólo use 3 columnas
```

DuckDB (column store, OLAP):

```
PARQUET_SCAN ventas.parquet
  Projection: año, ciudad, monto    ← solo 3 de 50 columnas
  Filters: año = 2025                ← pushdown al storage
  Row Groups Read: 2 / 8            ← saltó 6 row groups
```

El ecosistema columnar actual

Sistema	Qué lo distingue	Cuándo
DuckDB	embebido, local, sin servidor	análisis local
PostgreSQL	row store (la referencia OLTP)	apps, escrituras
Redshift	columnar en AWS	DW en AWS
BigQuery	serverless, paga por consulta	DW en GCP
Snowflake	separa cómputo / almacenamiento	multi-cloud
ClickHouse	velocidad en series temporales	eventos, logs
Databricks	lakehouse (Delta Lake)	Spark + SQL + ML

Todo salvo postgresQL: almacenamiento columnar, separación cómputo/almacenamiento y **Parquet** como estándar de almacenamiento — un Parquet de DuckDB lo lee BigQuery, Redshift o Spark sin convertir.

Resumen

Late materialization: filtrar columna por columna → *position lists*; reconstruir la fila solo en las posiciones que pasan todos los filtros.

Vectorización: procesar vectores de ~2048 valores (no una tupla a la vez) → SIMD, cache locality, sin overhead por tupla.

Procesamiento: las agregaciones se calculan sobre datos comprimidos (RLE/diccionario); los joins por hash descomprimen las llaves, podan con metadata y materializan tarde. Si los datos ya vienen ordenados, un **Sort-Merge Join** opera directo sobre los runs RLE; **z-ordering** ordena por varias dimensiones a la vez.

DuckDB: columnar embebido, sin servidor, lee Parquet/CSV directo; `EXPLAIN` muestra proyección de columnas + pushdown al storage.

